초급반 2주차

이O재 @ AlKor, CYDF

목차

- 복습
- 형 변환
- 연산자
- 조건문
- 연습문제

복습 - solved.ac

- solved.ac(솔브닥)이란?
  - 백준 유저들이 문제별 난이도 투표를 통해 어려운 문제/쉬운 문제를 구별
  - 난이도: 브론즈, 실버, 골드, 플래티넘, 다이아, 루비

(강사의 개인적인 생각)

- 브론즈 : 기초적인 문법만 사용해서 풀 수 있는 문제
- 실버 : 중학교 이상의 수학적/알고리즘적 지식을 사용해 풀 수 있는 문제
- 골드 : 고등학교 이상의 수학적/알고리즘적 지식을 사용해 풀 수 있는 문제
- 플래티넘: 대부분의 코딩테스트의 고난이도 문제
- 다이아 : 대기업 코딩테스트에 한 두 문제 나올 법한 난이도, 올림피아드 문제
- 루비 : 논문 수준의 지식을 가지고 푸는 문제

복습 - solved.ac

- 실력을 향상시키기 위해 난이도를 모르고 문제를 푸는 것이 바람직하나, 지키기 다소 어려움

- 세팅 방법
  - 설정 > solved.ac > 사용하기
  - 설정 > 보기 > solved.ac 티어 보기

복습 - 1001

```c
#include <bits/stdc++.h>

int main() {
    int a, b;

    scanf("%d %d", &a, &b);
    // scanf("%d", &a);
    // scanf("%d", &b);

    printf("%d\n", a - b);
}
```

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    int a, b;

    cin >> a >> b;
    cout << a - b << '\n';
}
```

복습 - 10998

```c
#include <bits/stdc++.h>

int main() {
    int a, b;

    scanf("%d %d", &a, &b);
    // scanf("%d", &a);
    // scanf("%d", &b);

    printf("%d\n", a * b);
}
```

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    int a, b;

    cin >> a >> b;
    cout << a * b << '\n';
}
```

복습 - 10869

```c
#include <bits/stdc++.h>

int main() {
    int a, b;

    scanf("%d %d", &a, &b);

    printf("%d\n", a + b);
    printf("%d\n", a - b);
    printf("%d\n", a * b);
    printf("%d\n", a / b);
    printf("%d\n", a % b);
}
```

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    int a, b;

    cin >> a >> b;

    cout << a + b << '\n'
         << a - b << '\n'
         << a * b << '\n'
         << a / b << '\n'
         << a % b << '\n';
}
```

복습 - 18108

```c
#include <bits/stdc++.h>

int main() {
    int a;

    scanf("%d", &a);

    printf("%d\n", a - 2541 + 1998);
}
```

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    int a;

    cin >> a;

    cout << a - 2541 + 1998 << '\n';
}
```

복습 - 10172

```c
#include <bits/stdc++.h>

int main() {
    printf("¦\\_/¦\n");
    printf("¦q p¦   /}\n");
    printf("( 0 )\"\"\"\\\n");
    printf("¦\"^\"`    ¦\n");
    printf("¦¦_/=\\\\__¦\n");
}
```

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    cout << "¦\\_/¦\n"
         << "¦q p¦   /}\n"
         << "( 0 )\"\"\"\\\n"
         << "¦\"^\"`    ¦\n"
         << "¦¦_/=\\\\__¦\n";
}
```

복습 - 1008

```c
#include <bits/stdc++.h>

int main() {
    float a, b;

    scanf("%f %f", &a, &b);

    printf("%.9f\n", a / b);
}
```

```
1 3
0.333333343
```

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    float a, b;

    cin >> a >> b;

    cout << fixed << setprecision(9);
    cout << a / b << '\n';
}
```

복습 - 1008

```c
#include <bits/stdc++.h>

int main() {
    double a, b;
    // long double a, b;

    scanf("%lf %lf", &a, &b);
    // scanf("%llf %llf", &a, &b);

    printf("%.16lf\n", a / b);
    // printf("%.16llf\n", a / b);
}

1 3
0.3333333333333333
```

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    double a, b;
    // long double a, b;

    cin >> a >> b;

    cout << fixed << setprecision(16);
    cout << a / b << '\n';
}
```

복습 - 2588

```c
#include <bits/stdc++.h>

int main() {
    int a, b;

    scanf("%d %d", &a, &b);

    printf("%d\n", a * (b % 10));
    printf("%d\n", a * ((b / 10) % 10));
    printf("%d\n", a * (b / 100));
    printf("%d\n", a * b);
}
```

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    int a, b;

    cin >> a >> b;

    cout << a * (b % 10) << '\n'
         << a * ((b / 10) % 10) << '\n'
         << a * (b / 100) << '\n'
         << a * b << '\n';
}
```

형 변환

- 데이터 타입을 다른 타입으로 바꾸는 것

- 형 변환이 필요한 경우
  - 1. 서로 다른 타입끼리 연산할 때

```
int     a = 10;
double  b = 3.5;
int     result1 = a + b; // 13
double  result2 = a + b; // 13.5
```

  - 2. 정밀도를 조절할 때

```
int     a = 5, b = 2;
double  result1 = a / b;          // 2
double  result2 = (double) a / b; // 2.5
```

연산자

- 증감 연산자

```
int a = 10;
a++; // a = 11
a--; // a = 10

++a; // a = 11
--a; // a = 10
```

- 전위 연산자: 변수의 값을 먼저 변화시키고 식에서 적용

```
int a = 10;
int b = (++a) + 1; // b = 12
```

- 후위 연산자: 식에서 먼저 적용하고 변수의 값을 변화

```
int a = 10;
int b = (a++) + 1; // b = 11
```

연산자

- Quiz. 아래 코드에서 b의 값은?

```
int a = 10;
int b = (a++) + (++a);
```

연산자

- Quiz. 아래 코드에서 b의 값은?

```
int a = 10;
int b = (a++) + (++a);
```

- Answer. 정의되지 않음 (컴파일러별로 결과값이 다르게 나타남)

- 컴파일러별로 a++를 먼저 계산할 수도, ++a를 먼저 계산할 수도 있음.
- 따라서 두 항을 분리해야 함.

연산자

- 관계 연산자

```
a == b // 같다
a != b // 같지 않다
a >  b
a <  b
a >= b
a <= b
```

- =과 ==의 비교
  - = : 대입 연산자, lhs에 rhs값을 저장한다.
  - ==: 비교 연산자, 두 값이 같으면 true(1), 다르면 false(0)를 반환

연산자

- 논리 연산자

```
<statement> && <statement>
<statement> || <statement>
!<statement>
```

- AND 연산자(&&): 두 statement가 모두 참(true)일 경우 참(true)

&&	statement1 == true	statement1 == false
statement2 == true	true	false
statement2 == false	false	false

연산자

- 논리 연산자

```
<statement> && <statement>
<statement> || <statement>
!<statement>
```

- OR 연산자(||): 두 statement 중 적어도 하나가 참(true)일 경우 참(true)

||	statement1 == true	statement1 == false
statement2 == true	true	true
statement2 == false	true	false

연산자

- 논리 연산자

```
<statement> && <statement>
<statement> || <statement>
!<statement>
```

- NOT 연산자(!): statement의 true, false 전환

!	statement == true	statement == false
result	false	true

연산자

- 조건 연산자(삼항 연산자)

```
<statement> ? <if_true> : <if_false>
```

- 사용 예시

```
int a = (10 % 2 == 0) ? 1 : 0; // a = 1
```

- 사용 이유: 코드를 간결하게 사용할 수 있음
  - 나중에 배울 조건문(if문)으로 대체하여 코드 작성 가능

연산자

- 비트 연산자

```
&  // AND
¦  // OR
^  // XOR
~  // NOT
<< // LEFT SHIFT
>> // RIGHT SHIFT
```

- AND 연산자(&)

```
char a = 10, b = 12;
char c = a & b; // c = 8

0000 1010
0000 1100
---------
0000 1000
```

연산자

- 비트 연산자

```
&  // AND
¦  // OR
^  // XOR
~  // NOT
<< // LEFT SHIFT
>> // RIGHT SHIFT
```

- OR 연산자(¦)

```
char a = 10, b = 12;
char c = a ¦ b; // c = 14

0000 1010
0000 1100
---------
0000 1110
```

연산자

- 비트 연산자

```
&  // AND
|  // OR
^  // XOR
~  // NOT
<< // LEFT SHIFT
>> // RIGHT SHIFT
```

- XOR 연산자(^)

```
char a = 10, b = 12;
char c = a ^ b; // c = 6

0000 1010
0000 1100
---------
0000 0110
```

연산자

- 비트 연산자

```
&  // AND
¦  // OR
^  // XOR
~  // NOT
<< // LEFT SHIFT
>> // RIGHT SHIFT
```

- NOT 연산자(~)

```
char a = 10;
char c = ~a; // c = -11

0000 1010
---------
1111 0101
```

연산자

- 비트 연산자

```
&  // AND
¦  // OR
^  // XOR
~  // NOT
<< // LEFT SHIFT
>> // RIGHT SHIFT
```

- LEFT SHIFT 연산자(<<)

```
char a = 10;
char c = a << 2; // c = 40

0000 1010
---------
0010 1000
```

연산자

- 비트 연산자

```
&  // AND
|  // OR
^  // XOR
~  // NOT
<< // LEFT SHIFT
>> // RIGHT SHIFT
```

- RIGHT SHIFT 연산자(>>)

```
char a = 10;
char c = a >> 2; // c = 2

0000 1010
---------
0000 0010
```

조건문

- 조건문(if문): 특정 조건에 따라 실행할지, 말지를 결정하는 문법

```
if (<statement> == true) {
    code
}
```

- 조건에는 주로 논리연산자로 이루어진 식이 쓰임

```
int a = 10;

if (a % 2 == 0) {
    printf("a는 짝수입니다.\n");
}

// a는 짝수입니다.
```

조건문

- if문은 여러 개 사용해도 별개로 작동함

```
int a = 10;

if (a % 2 == 0) {
    printf("a는 짝수입니다.\n");
}

if (a % 2 != 1) {
    printf("a는 홀수가 아닙니다.\n");
}

// a는 짝수입니다.
// a는 홀수가 아닙니다.
```

조건문

- 조건이 거짓일 경우 사용하는 else

```
int a = 10;

if (a % 2 == 0) {
    printf("a는 짝수입니다.\n");
} else {
    printf("a는 홀수입니다.\n");
}

// a는 짝수입니다.
```

조건문

- if – else는 하나의 세트임

```
int a = 10;

if (a % 2 == 0) {
    printf("a는 짝수입니다.\n");
} else {
    printf("a는 홀수입니다.\n");
}

if (a % 3 == 0) {
    printf("a는 3의 배수입니다.\n");
} else {
    printf("a는 3의 배수가 아닙니다.\n");
}

// a는 짝수입니다.
// a는 3의 배수가 아닙니다.
```

조건문

- 분기를 2갈래 뿐 아니라 3갈래, 4갈래로 나눌 수 있음

```
int a = 10;

if (a % 2 != 0) {
    printf("a는 홀수입니다.\n");
} else {
    if (a == 5) {
        printf("a는 5입니다.\n");
    } else {
        printf("a는 5가 아닙니다.\n");
    }
}

// a는 5가 아닙니다.
```

조건문

- 갈래를 더욱 나눌 경우 들여쓰기가 많이 이루어짐 ➔ else if 사용

```
int a = 10;

if (a % 2 != 0) {
    printf("a는 홀수입니다.\n");
} else if (a == 5) {
    printf("a는 5입니다.\n");
} else {
    printf("a는 5가 아닙니다.\n");
}

// a는 5가 아닙니다.
```

- else 안의 if를 위로 끌어와서 쓰는 느낌

조건문

- 점수를 입력받아서, 아래의 조건을 만족하도록 출력하는 코드 작성
  - 95~100: A+
  - 90~94 : A
  - 85~89 : B+
  - 80~84 : B
  - 75~79 : C+
  - 0~74 : 사람이 아님
  - 그 외 : 잘못 입력함

조건문

```
#include <bits/stdc++.h>

int main() {
    int a;
    scanf("%d", &a);

    if (95 <= a <= 100) {
        printf("A+\n");
    } else if (90 <= a < 95) {
        printf("A\n");
    } else if (85 <= a < 90) {
        printf("B+\n");
    } else if (80 <= a < 85) {
        printf("B\n");
    }
```

```
    else if (75 <= a < 80) {
        printf("C+\n");
    } else if (70 <= a < 75) {
        printf("사람이 아님\n");
    } else {
        printf("잘못 입력함\n");
    }
}
```

조건문

- 왜 틀린 코드인가?

```
int a = 110;

if (95 <= a <= 100) { ... }

95 <= a(110) ➔ true(1)

95 <= a <= 100 ➔ (95 <= a) <= 100 ➔ 1 <= 100 ➔ true
```

- 따라서 점수로 110점을 입력해도 “잘못 입력함”이 아닌, A+가 출력됨

조건문

```
#include <bits/stdc++.h>

int main() {
    int a;
    scanf("%d", &a);

    if (95 <= a && a <= 100) {
        printf("A+\n");
    } else if (90 <= a && a < 95) {
        printf("A\n");
    } else if (85 <= a && a < 90) {
        printf("B+\n");
    } else if (80 <= a && a < 85) {
        printf("B\n");
    }
```

```
    else if (75 <= a && a < 80) {
        printf("C+\n");
    } else if (70 <= a && a < 75) {
        printf("사람이 아님\n");
    } else {
        printf("잘못 입력함\n");
    }
}
```

조건문

```
#include <bits/stdc++.h>

int main() {
    char a;
    scanf("%c", &a);

    if (a < 0 || a > 100) {
        printf("잘못 입력함\n");
    } else if (a >= 95) {
        printf("A+\n");
    } else if (a >= 90) {
        printf("A\n");
    } else if (a >= 85) {
        printf("B+\n");
    }
```

```
    else if (a >= 80) {
        printf("C+\n");
    } else if (a >= 75) {
        printf("사람이 아님\n");
    }
}
```

연습문제

- 2주차 문제집

- 수업시간에 풀 문제: 1330, 9498, 28691
- 숙제: 2753, 14681, 2884, 2480, 27294

- Optional: 2525